Lab3-hw3

Megan Hessel

Overview

In this lab we'll collect posts from Bluesky Social using the `{atrrr}` package and apply lexicon-based sentiment analysis tools — the same tools we covered in lecture (VADER via `{vader}`, `{sentimentr}`, and `{tidytext}` lexicons). By the end you'll have a working pipeline from API call to sentiment scores, and you'll have a concrete sense of where these tools succeed and where they fall short. We'll walk through some of this in class with sample data, then you'll be responsible for completing it with your own data set for the assignment.

Setup

1. Create a Bluesky account and app password (you'll do this your assignment)

```
# Add bluesky account information/ credientials in to the `.Renviron`
# usethis::edit_r_environ()
```

2.Packages

```
#install.packages(c("atrrr", "tidyverse", "tidytext", "sentimentr", "remotes", "textdat
a"))

# tidyvader isn't on CRAN:
#install.packages("devtools")
#devtools::install_github("chris31415926535/tidyvader")
```

```
library(atrrr)
library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✔ dplyr     1.2.1     ✔ readr     2.1.6
## ✔ forcats   1.0.1     ✔ stringr   1.6.0
## ✔ ggplot2   4.0.2     ✔ tibble    3.3.1
## ✔ lubridate 1.9.5     ✔ tidyr     1.3.2
## ✔ purrr     1.2.1
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts
to become errors
```

```r
library(tidytext)
library(sentimentr)
library(remotes)
library(textdata)
```

Part 1: Authenticate and Pull Posts

```r
# Authenticate
# Catches token so wont't get hit login rate limits
# auth(
#   user = Sys.getenv("BSKY_USER"),
#   pass = Sys.getenv("BSKY_PASS")
#   )

# Input the term and limit
raw_posts <- search_post("ecosystem service", limit = 1000) #you'll do this later
```

```
## ℹ Found an *unknown total* of posts that fit the query
```

```
## ⠔ Got 100 posts, but there is more.. [550ms]⠔ Got 199 posts, but there is more.. [742
ms]⠔ Got 298 posts, but there is more.. [949ms]⠔ Got 396 posts, but there is more.. [1.3
s]⠔ Got 494 posts, but there is more.. [1.5s]⠔ Got 593 posts, but there is more.. [1.7
s]⠔ Got 688 posts, but there is more.. [2s]⠔ Got 786 posts, but there is more.. [2.2s]
⠔ Got 879 posts, but there is more.. [2.4s]⠔ Got 974 posts, but there is more.. [2.6s]⠔
Got 1067 posts, but there is more.. [2.8s]                                              ℹ
Parsing 1067 results.✔ Got 1067 results. All done!
```

Investigate results!

```r
#glimpse(raw_posts)
#head(raw_posts)
```

Part 2: Clean and Prepare Text

Pull out necessary columns & basic cleaning

```
posts <- raw_posts %>%
  select(uri, indexed_at, text, like_count, reply_count, repost_count) %>%
  mutate(
    indexed_at = as.POSIXct(indexed_at), # who can tell me how to
    post_date  = as.Date(indexed_at),
    text_clean = str_remove_all(text, "https?\\S+"), # remove URLs
    text_clean = str_remove_all(text_clean, "@\\S+"),# remove mentions
    text_clean = str_replace_all(text_clean, "\u2019", "'"), # normalize all curly ' to
R's '
    text_clean = str_remove_all(text_clean, "'s\\b"), # remove possessives
    text_clean = str_squish(text_clean) # cleans all the added white space
  ) %>%
  filter(nchar(text_clean) > 10)
```

Number of posts:

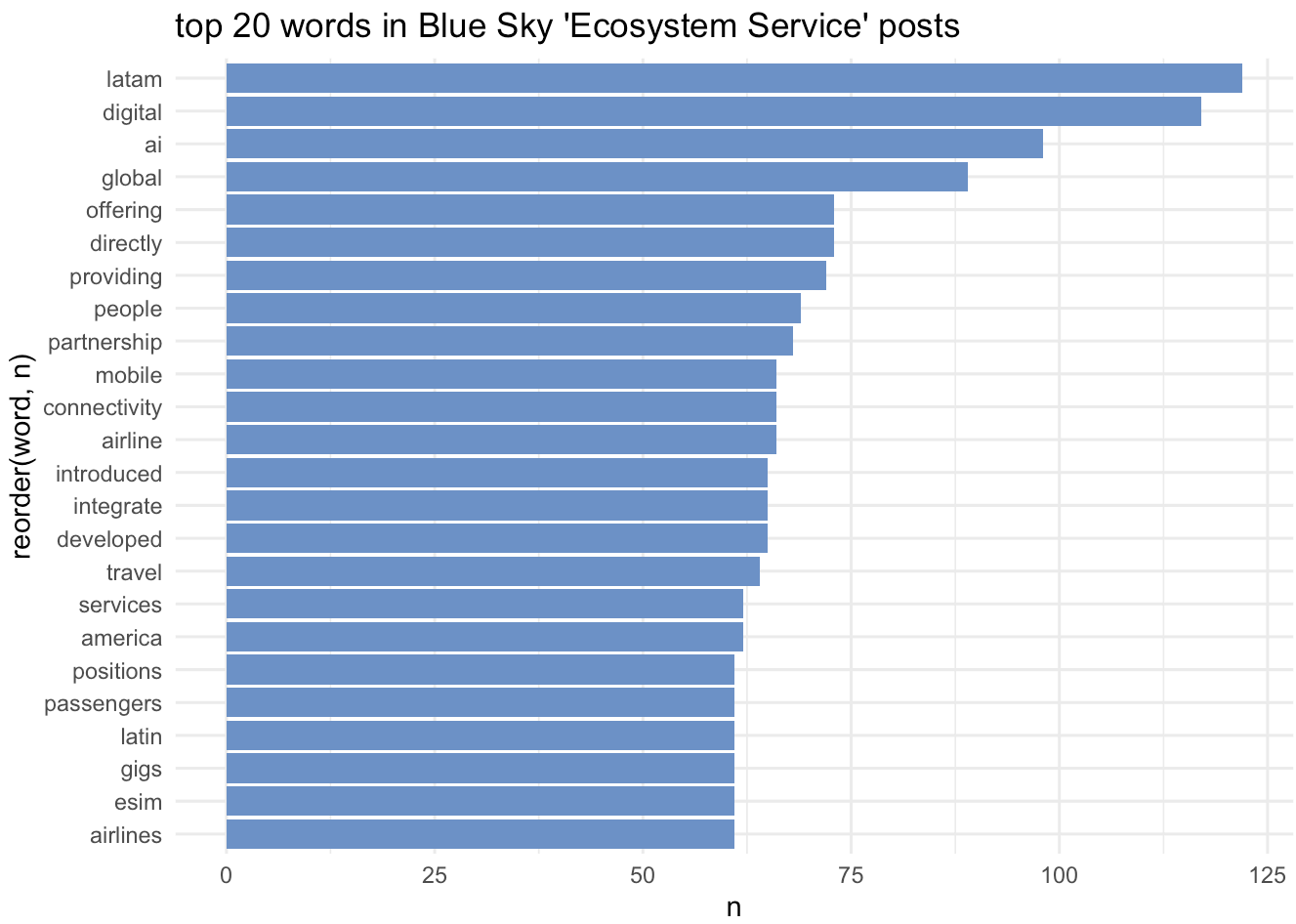
```
nrow(posts)
```

```
## [1] 1060
```

Part 3: Word Frequency

Explore and plot more frequent words used

```r
# Standard stop words
data("stop_words")

tokens_clean <- posts |>
  unnest_tokens(word, text_clean) |>
  anti_join(stop_words, by = "word")

# Custom stop words
custom_sw <- tibble(word = c(
  # search terms
  "ecosystem", "service"
  # other common but uninformative text
  # I can't think of anything
))

tokens_clean2 <- tokens_clean %>%
  anti_join(custom_sw, by = "word")

# Word frequency plot

word_counts <- tokens_clean2 %>%
  count(word, sort = TRUE)

word_counts %>%
  slice_max(n, n =20) %>%
  ggplot(aes(x = reorder(word, n), y = n)) +
  geom_col(fill = "#6c91c9") +
  coord_flip() +
  labs(title = "top 20 words in Blue Sky 'Ecosystem Service' posts") +
  theme_minimal()
```



Part 4: Sentiment Analysis

> **Note:** If you need to re-run any chunk in this section, run `rm(posts)` in the console first and re-run from Part 2. This avoids duplicate column errors.

4a. Bing lexicon (word-level)

The Bing lexicon is a polarity / binary lexicon grouping where yes/no = positive/negative. It is a list of words and their positive or negative sentiments.

```
# Load lexicon
bing_lexicon <- get_sentiments("bing")
#get_sentiments("bing")
```

Join the bing lexicon list to the tokenized the posts and calcultae the net score (positive word count minus negative word count) per post.

```
# Score posts with Bing
bing_scores <- posts %>%
  # Tokenize posts
  unnest_tokens(word, text_clean) %>%
  # Join the words to the sentiment lexicon
  inner_join(bing_lexicon, by = "word") %>%
  count(uri, sentiment) %>%
  pivot_wider(names_from = sentiment,
              values_from = n,
              values_fill = 0) %>%
  # Calculate total sentiment score: positives - negative
  mutate(score_b = positive - negative)
```

Join the scores back onto `posts`

```
# Join scores to posts
posts <- posts %>%
  left_join(bing_scores, "uri") %>%
  mutate(bing_score = replace_na(score_b, 0)) # NA scores = 0

# Check if it joined the score column
colnames(posts)
```

```
##  [1] "uri"          "indexed_at"   "text"         "like_count"   "reply_count"
##  [6] "repost_count" "post_date"    "text_clean"   "negative"     "positive"
## [11] "score_b"      "bing_score"
```

4b. AFINN lexicon (numeric scores)

Score the posts with the afinn sentiment package where each word carries an integer value (–5 to +5).

```
# You will be prompted to download the AFINN lexicon the first time —
# run this line in the console and select Yes, then it will be cached locally
afinn <- get_sentiments("afinn")

# Score text
afinn_scores <- posts %>%
  # Tokenize posts
  unnest_tokens(word, text_clean) %>%
  # Inner Join the words to the sentiment lexicon
  inner_join(afinn, by = "word") %>%
  # Score: sum of afinn scores for each URI
  group_by(uri) %>%
  summarise(score_af = sum(value), .groups = "drop")


# Join scores to posts
posts <- posts %>%
  left_join(afinn_scores, "uri") %>%
  mutate(afinn_score = replace_na(score_af, 0)) # NA scores = 0

# str(posts) # Checking dataset
```

Clean the dataset before moving on

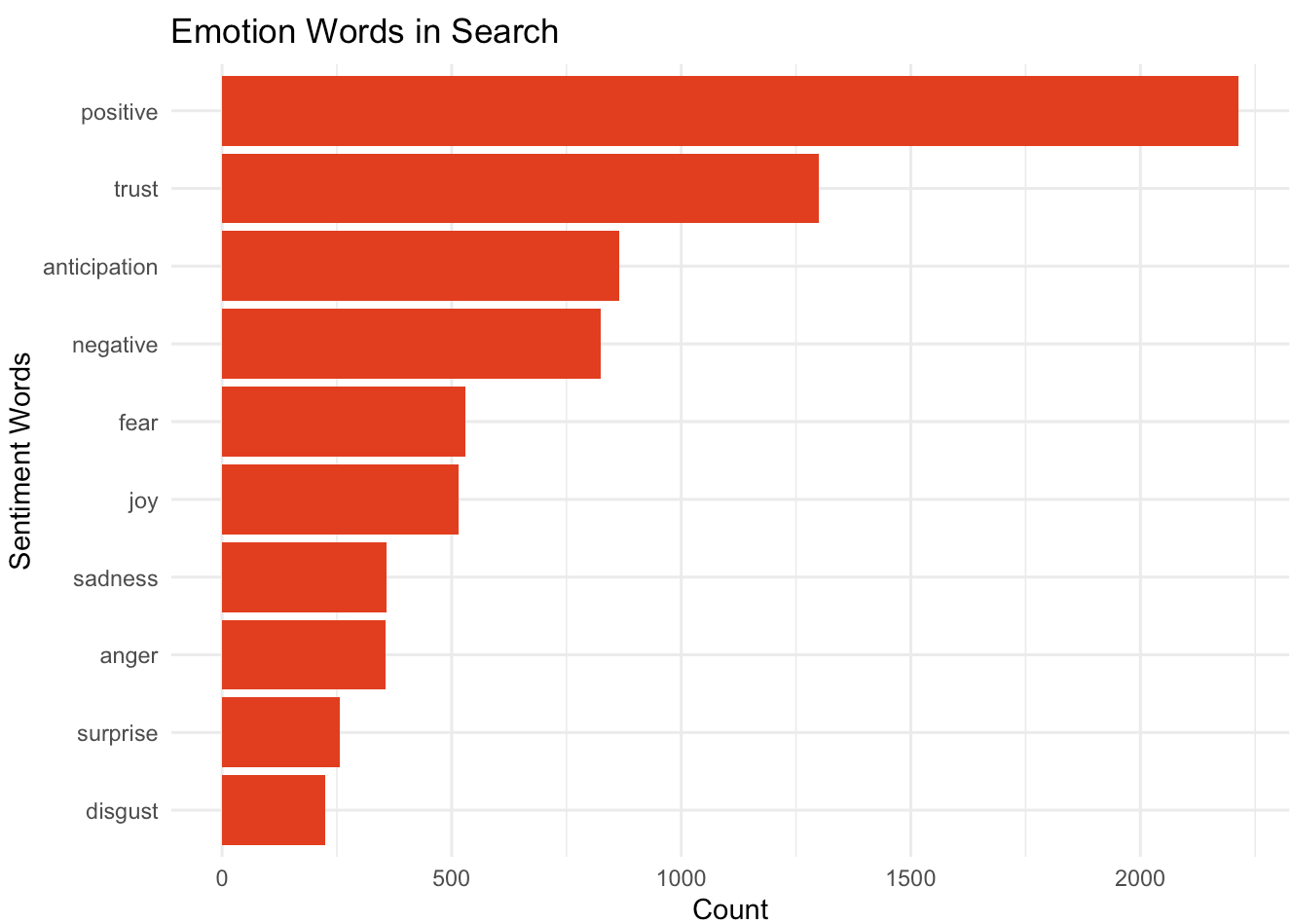
```
posts <- posts %>%
  select(-c(negative, positive, score_b, score_af))
```

4c. NRC lexicon (emotion categories)

The NRC lexicon tags words with 8 emotions (anger, fear, anticipation, trust, surprise, sadness, joy, disgust) plus positive/negative.

Now, count emotion words per post and visualize which emotions dominate the corpus.

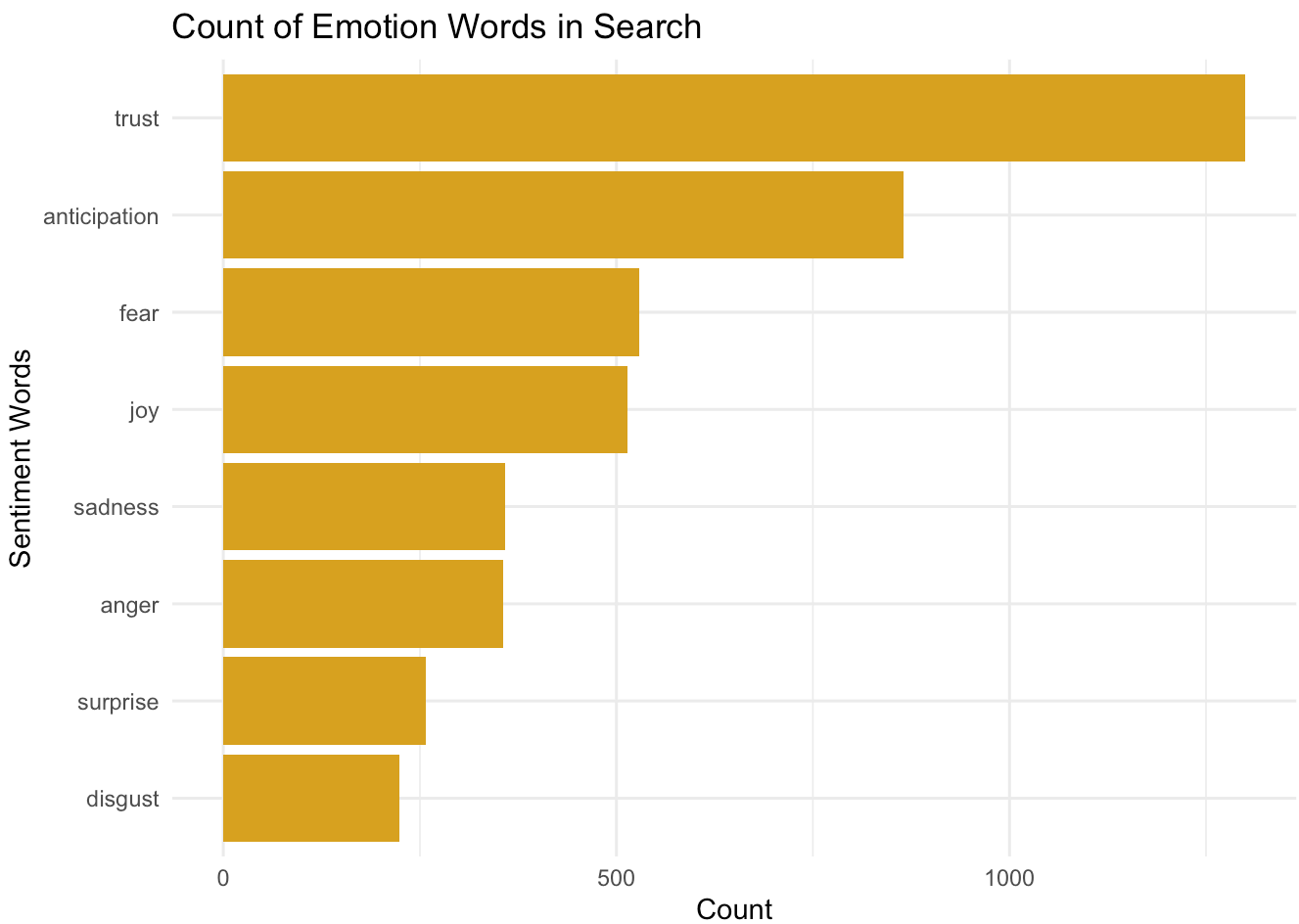
```r
nrc <- get_sentiments("nrc")

# Score text - Count emotion words per post
nrc_scores <- posts %>%
  # Tokenized posts
  unnest_tokens(word, text_clean) %>%
  # Join to NRC
  inner_join(nrc, by = "word", relationship = "many-to-many") %>%
  # Count emotion words per post
  group_by(uri, sentiment) %>%
  summarise(nrc_count = n(), .groups = "drop")


# Visualize which emotions dominate the corpus
nrc_scores %>%
  group_by(sentiment) %>%
  summarise(nrc_count = sum(nrc_count), .groups = 'drop') %>%
  ggplot() +
  geom_col(aes(y = reorder(sentiment, nrc_count), x = nrc_count),
           fill = "#e14225") +
  theme_minimal() +
  labs(title = "Emotion Words in Search", x = "Count", y = "Sentiment Words")
```



```
#Join to post
posts <- posts %>%
  left_join(nrc_scores, by = "uri") %>%
  rename(nrc_sentiment_words = sentiment)
```

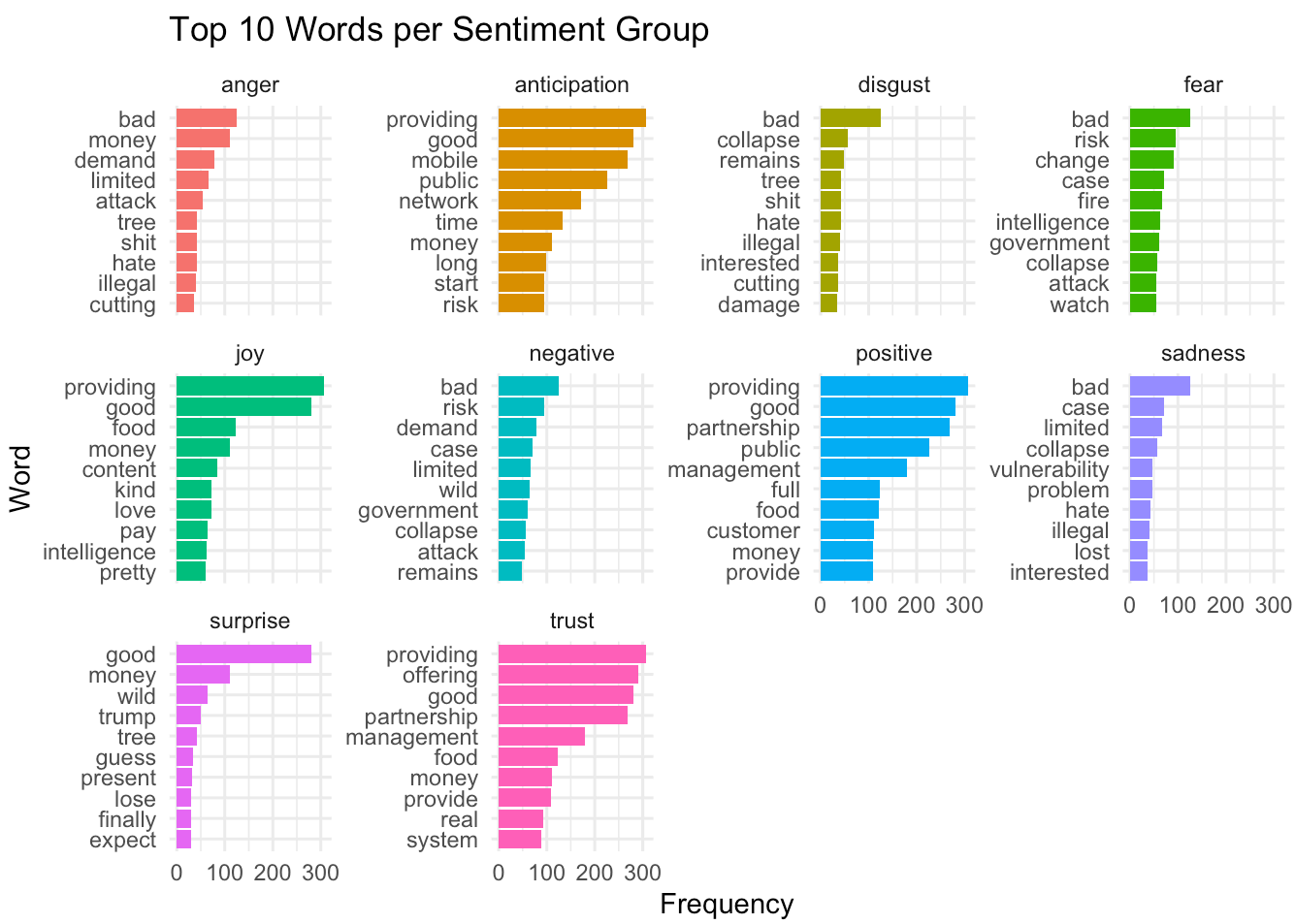
Which emotions dominate the corpus? Plot the count of total scored words for each of the 8 emotions.

```
nrc_scores %>%
  group_by(sentiment) %>%
  summarise(nrc_count = sum(nrc_count)) %>%
  filter(!sentiment %in% c("positive", "negative")) %>% # Remove positive and negative
  ggplot() +
  geom_col(aes(y = reorder(sentiment, nrc_count), x = nrc_count), fill = "goldenrod") +
  theme_minimal() +
  labs(title = "Count of Emotion Words in Search", x = "Count", y = "Sentiment Words")
```



What words are driving each emotion? Create plots of the top 10 words for each emotion (hint: ggplot and `facet_wrap(~sentiment)`).

```r
# Find frequency of words for each sentiment
sentiment_word_count <- posts %>%
  unnest_tokens(word, text_clean) %>% # Tokenized posts
  inner_join(nrc, by = "word", relationship = "many-to-many") %>% # Join to NRC
  count(sentiment, word, sort = TRUE)  # Count of words per sentiment

# Get the top 10 most frequent words for each sentiment group
top_10 <- sentiment_word_count %>%
  group_by(sentiment) %>%
  slice_max(order_by = n, n = 10, with_ties = FALSE) %>%
  ungroup()

# Plot the top 10 words for each sentiment group
ggplot(top_10, aes(x = reorder_within(word, n, sentiment), y = n, fill = sentiment)) +
  geom_col(show.legend = FALSE) +
  scale_x_reordered() +  # Keeps ordering within facets
  facet_wrap(~ sentiment, scales = "free_y") +
  labs(
    title = "Top 10 Words per Sentiment Group",
    x = "Word",
    y = "Frequency"
  ) +
  coord_flip() +
  theme_minimal()
```



4d. sentimentr (sentence-aware)

Unlike the word-level lexicons above, `{sentimentr}` scores at the sentence level and handles valence shifters like negation — "not alarming" won't score the same as "alarming." Pass the full `text_clean` vector to `sentiment_by()`. Rename the resulting `ave_sentiment` to `sr_scores` and join back onto `posts` by row index. I'll give you this code as we haven't fully covered this yet.

```
# Using sentimentr
sr_scores <- sentiment_by(posts$text_clean) %>%
  select(element_id, ave_sentiment) %>%
  rename(sr_score = ave_sentiment) %>%
  mutate(element_id = as.integer(element_id))
```

```
## Warning: Each time `sentiment_by` is run it has to do sentence boundary disambiguatio
n when a
## raw `character` vector is passed to `text.var`. This may be costly of time and
## memory.  It is highly recommended that the user first runs the raw `character`
## vector through the `get_sentences` function.
```

```
# Join sr score back into posts by row index
posts <- posts %>%
  mutate(element_id = row_number()) %>%
  left_join(sr_scores, by = "element_id")
```

4e. VADER

VADER scores the full post text (no tokenization needed) and returns four values: `pos`, `neu`, `neg`, and `compound`. Use `tidyvader::vader()` on the `text_clean` column — it mutates the result directly onto the dataframe. Keep only the `compound` score and drop the component columns.

Let's run VADER on a single post manually before applying it to the whole dataset:

```
# Look at a few posts
# posts %>% select(text_clean) %>% slice(1:10)

# run VADER on one post — swap in whichever row looks interesting to you
posts$text_clean[3]
```

```
## [1] "[4/5] service capabilities of intermediaries. Promoting the high-quality develop
ment of listed companies is a systematic project. All regions and departments must take
the initiative to optimize the business environment, jointly create a good ecosystem tha
t supports the high-quality"
```

```
tidyvader::vader_chr(posts$text_clean[3])
```

```
##   compound    pos    neu neg
## 1   0.9042 0.2857 0.7143   0
```

What did VADER catch? What did it miss? How does the compound score relate to the word-level scores?

VADAR determined the whole text was neutral (neu = 1), which makes sense as the sentence is a factual, descriptive statement with no emotional language.

Now score all the posts with VADER. Rename it's default scoring column to `vader_score`.

```
posts <- posts %>%
  mutate(vader_score = purrr::map_dbl(text_clean, ~ tidyvader::vader_chr(.x)$compound))
```

Part 5: Compare the Methods

Correlations

Let's see to what extent the 4 numerical methods (bing, afinn, sr, and vader) are in agreement. Select the 4 scores and use `cor()` using "complete.obs".

```
posts %>%
  select(bing_score, afinn_score, sr_score, vader_score) %>% # Select 4 scores
  cor(use = "complete.obs") # use cor()
```

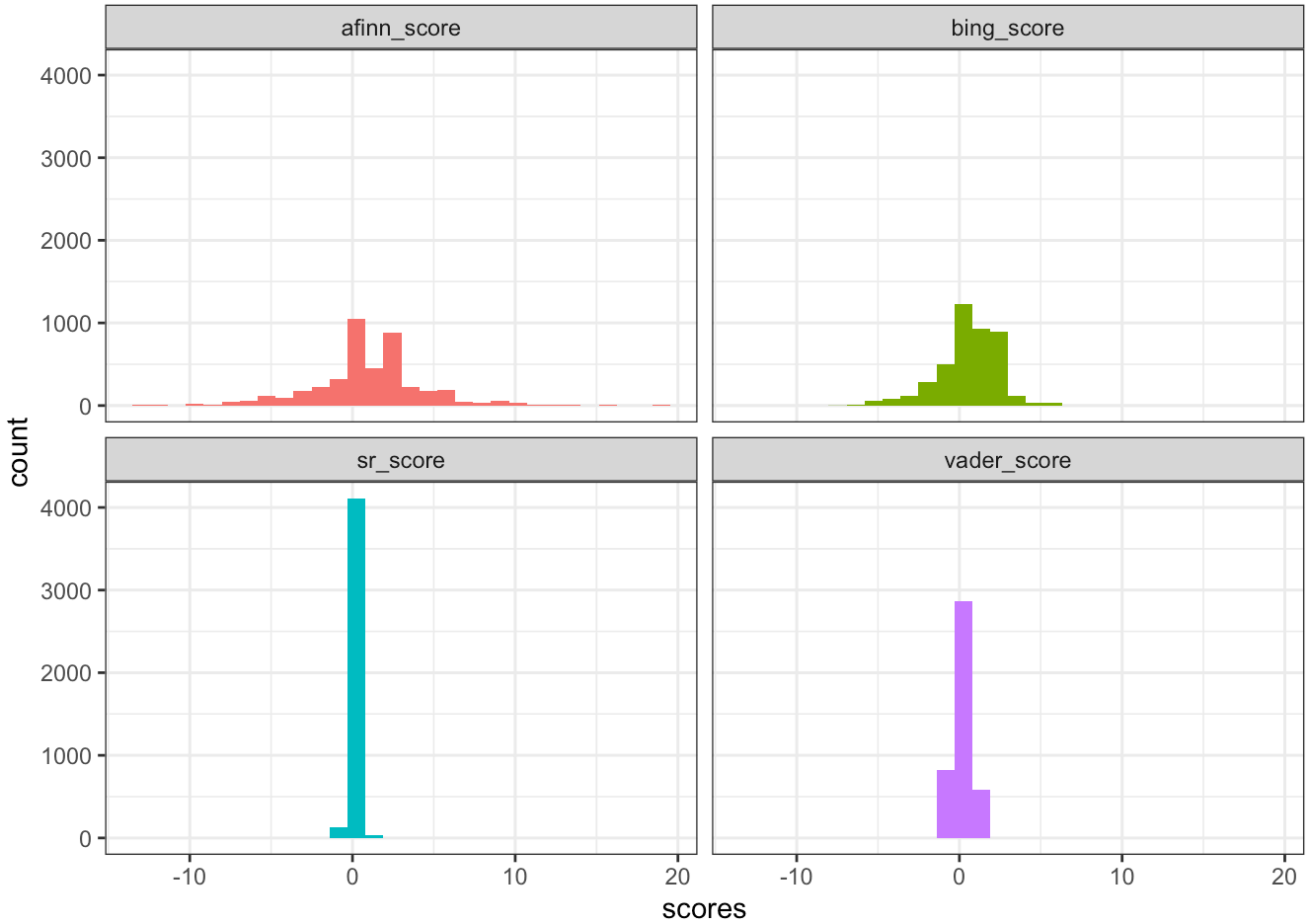
```
##             bing_score afinn_score  sr_score vader_score
## bing_score   1.0000000   0.6931579 0.6006261   0.6634219
## afinn_score  0.6931579   1.0000000 0.5079764   0.7735428
## sr_score     0.6006261   0.5079764 1.0000000   0.5563381
## vader_score  0.6634219   0.7735428 0.5563381   1.0000000
```

Score distributions

Use `pivot_longer()` to reshape the four score columns into a single `score` column with a `method` identifier, then plot with `geom_histogram()` and `facet_wrap()`.

```
posts %>%
  pivot_longer(cols = c(bing_score, afinn_score, sr_score, vader_score),
               names_to = "method",
               values_to = "scores") %>%
  ggplot() +
  geom_histogram(aes(scores, fill = method), show.legend = FALSE) +
  facet_wrap(~method) +
  theme_bw()
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



Question 1: Do the four methods broadly agree or disagree? Where do you see the biggest differences in the distributions? Which method produces the most zeros, and why?

All four methods show a neutral corpus.

The VADAR and sentimentr results are similar. Their scores are very neutral, having high counts at/around 0. These are both sentence-level analyzers that account for context. These methods are more precise but more conservative as they miss sentiment that is not explicitly emotional. The accumulated text does not contain strong language, so VADAR and sentimentr scores are concentrated at 0.

Unlike VADAR and sentimentr, the bing and afinn scores both have a large score range and broader distributions. Bing and afinn are word-count based so they accumulate scores across a document, resulting in more variations across posts.

Part 6: Spot-Check for Misclassification

Find posts where the methods disagree most — these are the best candidates for understanding failure modes. Use `mutate()` to create binary flags from the continuous scores, then `filter()` for posts where the two methods point in opposite directions.

Finding where bing and VADAD scores point opposite directions.

```
misclassifications <- posts %>%
  select(c(text_clean, bing_score, vader_score)) %>%
  # New column with score signs
  mutate(bing_flag = sign(bing_score),
         #afinn_flag = sign(afinn_score),
         #sr_flag = sign(sr_score),
         vadar_flag = sign(vader_score)
         ) %>%
  # Filter for posts where two methods point in opposite directions
  # Picking bing and VADAR
  filter((bing_flag == -1 & vadar_flag == 1) |
         (bing_flag == 1 & vadar_flag == -1))

head(misclassifications)
```

```
## # A tibble: 6 × 5
##   text_clean                          bing_score vader_score bing_flag vadar_flag
##   <chr>                                    <int>       <dbl>     <dbl>      <dbl>
## 1 I'm not sure it makes up losses l…           1      -0.287         1         -1
## 2 I'm not sure it makes up losses l…           1      -0.287         1         -1
## 3 I'm not sure it makes up losses l…           1      -0.287         1         -1
## 4 Basically use Gamepass as a promo…          -1       0.226        -1          1
## 5 Basically use Gamepass as a promo…          -1       0.226        -1          1
## 6 Basically use Gamepass as a promo…          -1       0.226        -1          1
```

To rank posts by total disagreement, compute the absolute difference between the two scores with `abs()`, then `arrange(desc(...))` to surface the highest-conflict posts.

```
high_conflict_text <- posts %>%
  # Text per row and keep only neccessary columns
  select(c(text_clean, bing_score, vader_score)) %>%
  group_by(text_clean) %>%
  summarise(bing_score = mean(bing_score),
            vader_score = mean(vader_score)) %>%
  # New column with score signs
  mutate(bing_flag = sign(bing_score),
         #afinn_flag = sign(afinn_score),
         #sr_flag = sign(sr_score),
         vadar_flag = sign(vader_score)
         ) %>%
  # Filter for posts where two methods point in opposite directions
  # Picking bing and VADAR
  filter((bing_flag == -1 & vadar_flag == 1) |
         (bing_flag == 1 & vadar_flag == -1)) %>%
  # Compute the absolute difference between the two scores
  mutate(abs_difference = abs(bing_score) - abs(vader_score)) %>%
  # Arrange for highest-conflict first
  arrange(desc(abs_difference))

head(high_conflict_text)
```

```
## # A tibble: 6 × 6
##   text_clean          bing_score vader_score bing_flag vadar_flag abs_difference
##   <chr>                    <dbl>       <dbl>     <dbl>      <dbl>          <dbl>
## 1 Its an ecosystem. …          6     -0.151          1         -1           5.85
## 2 14) Kosher wedding…          3     -0.0687         1         -1           2.93
## 3 this suspect n/c i…         -3      0.209         -1          1           2.79
## 4 Urgent security up…         -3      0.310         -1          1           2.69
## 5 tanstack.com/ai/la…          3     -0.422          1         -1           2.58
## 6 Honestly shocked a…         -3      0.643         -1          1           2.36
```

Question 2: Read through 5 of the high-conflict posts. For each one, write a sentence explaining which method you think is closer to correct and why.

Row 1: Bing is closer. The post is frustrated/critical but Bing catches the positive framing around the industry. VADER's negative score misses that the negativity is directed at policymakers, not the subject.

Row 2: Bing is closer. Mixed sentiment, net positive is reasonable. VADER going negative likely triggered on "shit" without capturing the overall constructive tone.

Row 3: VADER is closer. The post is clearly critical/negative. Bing scoring +3 likely matched words like "proud," "massive," "peacemaker" without understanding they're used sarcastically or negatively.

Row 4: Neither is great, but VADER is wrong. Scoring +0.31 on an urgent security warning is clearly off. Bing's -3 at least gets the direction right, likely from "vulnerability," "flaw," "denial."

Row 5: Bing is closer. The post is promotional/positive. VADER scoring -0.42 likely penalized "no middleman, no fees, no lock-in" treating the negations as negative sentiment rather than positive selling points.

Question 3:

Based on everything you've run, which method would you choose if you were delivering a report onpublic sentiment around your topic, and why? What additional steps would you take to validate your choice?

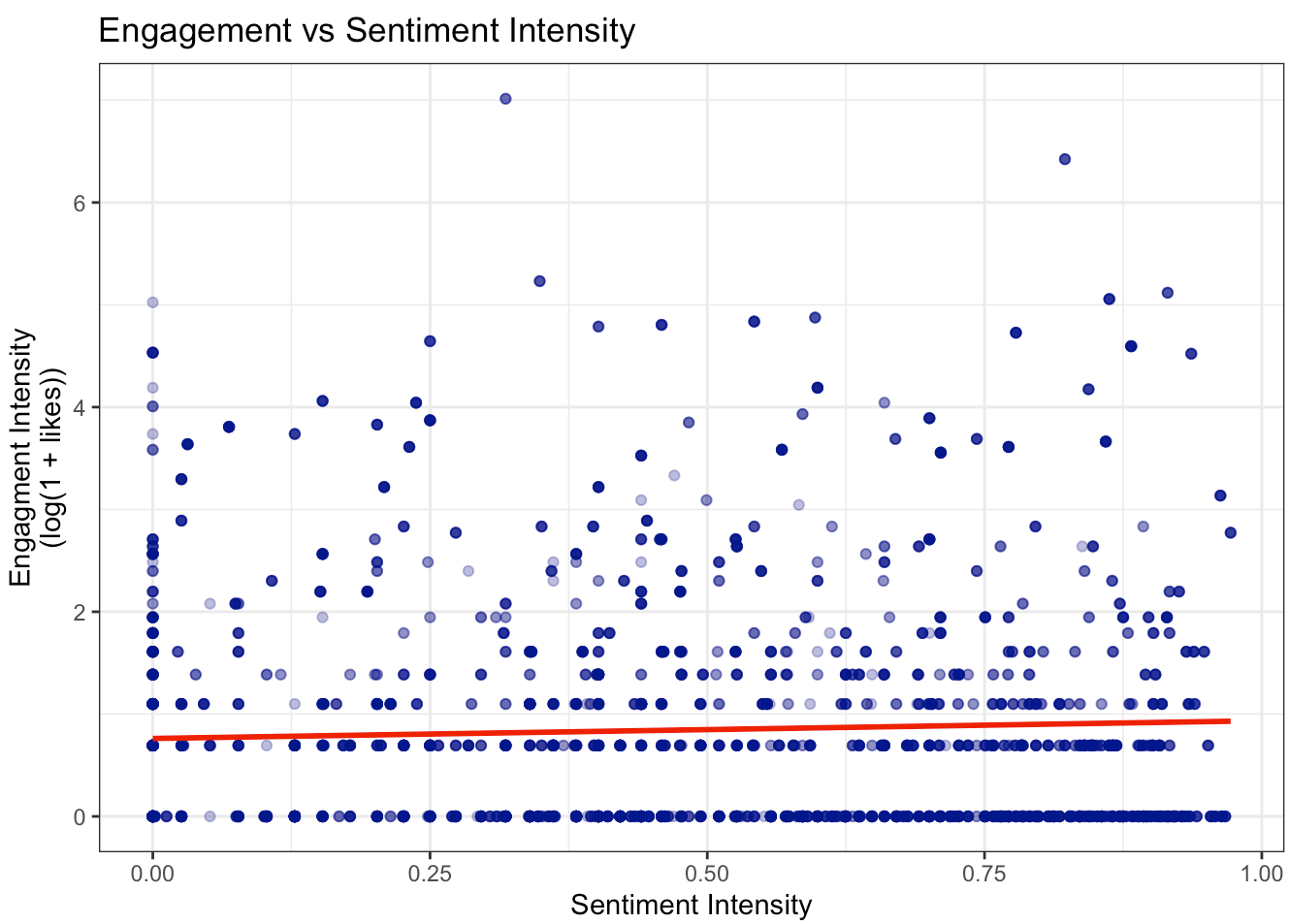
I would choose VADAR for this corpus. The posts are short, social media style text which is VADAR's strengths. VADAR handles negation and context better than bing and afinn. SentimentR is also context-aware but produces even more zeros, losing too much signal on this data. To validate, I would randomly sample posts as positive, neutral, and negative and look at results from each method's classifications. I would specifically look at high-engagement posts and see if VADAR scores those those correctly.

Bonus: Engagement vs. Sentiment (completion optional)

Do more emotionally charged posts get more engagement? Use `abs()` on the VADER compound score to capture emotional intensity regardless of direction, apply `log1p()` to `like_count` to handle the skewed distribution, and add `geom_smooth(method = "lm")` to visualize the trend.

```
posts %>%
    mutate(
    sentiment_int = abs(vader_score), # Capture emotional intensity
    likes_log = log1p(like_count) # Handle skewed distribution
  ) %>%
  # Plot
  ggplot(aes(x = sentiment_int, y = likes_log)) +
  geom_point(alpha = 0.3, color = "darkblue") +
  geom_smooth(method = "lm", se = FALSE, color = "red2") +
  labs(
    title = "Engagement vs Sentiment Intensity",
    x = "Sentiment Intensity",
    y = "Engagment Intensity\n(log(1 + likes))") +
  theme_bw()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Bonus: Is there a relationship between sentiment intensity and engagement? What does that suggest about what gets amplified on Bluesky?

There is a very weak positive trend between sentiment intensity and engagement. Sentimental intensity cannot predict likes (engagment) alone. Interestingly, there is high clusters at low engagement regardless of intensity. This suggests Bluesky amplification and engagement is driven by factors other than sentiment intensity.